JobMatch — Project Analysis Report

Resume & Job Skill Matcher

Status: Complete / Portfolio Ready

1. Executive Summary

JobMatch is a Flask-based resume-to-job matching web application that extracts meaningful requirements, evaluates resume evidence, applies weighted scoring, identifies strong/partial/missing evidence, generates recommendations, and stores analysis history in SQLite. The final portfolio version is intentionally frozen at the stable working implementation and does not require a paid AI API.

2. Problem Statement

Candidates often compare resumes and job descriptions manually. JobMatch was built to make requirement coverage, evidence strength, important gaps, and improvement areas easier to understand through an explainable workflow.

3. Objectives

Accept resume/JD PDFs or text; extract useful text; identify requirements and skills; compare evidence; apply weighted scoring; classify strong, related/partial, and missing evidence; generate recommendations; show project alignment; and maintain analysis history.

4. Technology Stack

Python, Flask, pypdf, SQLite, Jinja2, HTML/CSS/JavaScript, JSON, and regular-expression based text normalization.

5. Core Architecture

Input → PDF/text extraction → normalization → resume evidence extraction + JD requirement extraction → requirement classification → evidence comparison → weighted scoring → gap analysis → recommendations → SQLite history → UI.

6. Major Modules

Document input, PDF extraction, skill extraction, requirement classification, evidence evaluation, weighted scoring, recommendation generation, project alignment, and SQLite-backed history.

7. Evidence Model

Concrete project and internship evidence is treated more strongly than a simple skills-list mention. The goal is to evaluate where and how a requirement is evidenced rather than only whether a keyword appears.

8. Scoring Model

The system calculates overall match, technical fit, responsibility fit, professional fit, skill coverage, core score, supporting score, matched weight, and total requirement weight. Higher-priority requirements have greater influence.

9. User Interface

The application uses a dark SaaS-style interface with analyzer, history, and detailed-result views. Results include score visualization, coverage, strong matches, partial/related evidence, missing requirements, and recommendations.

10. Database

SQLite stores analysis history including skills, match information, scores, timestamps, requirement details, and active analysis state.

11. Testing

Development testing covered application startup, PDF extraction, skill and requirement extraction, weighting, classification, SQLite history, details-page data handling, UI behavior, and realistic Software Engineer job-description scenarios.

12. Engineering Lessons

Keyword presence is not expertise; context matters; missing evidence is not proof of inability; job descriptions contain noise; and an explainable result is more useful than an unexplained percentage.

13. Limitations

The application is not a hiring predictor. It cannot determine selection probability. Deterministic semantic coverage is limited by its taxonomy and text quality, and the score represents resume/JD alignment rather than real-world engineering ability.

14. AI Exploration

An experimental semantic AI layer was explored, but it was not made a dependency of the final portfolio release because it required external API credits. The stable release remains independently demonstrable.

15. Security

The repository must not contain API keys, secret .env files, virtual environments, cache files, or local databases containing user data.

16. Portfolio Value

The project demonstrates Python, Flask/backend development, PDF processing, text processing, SQL/SQLite, algorithmic scoring, requirement analysis, frontend integration, debugging, and explainable application design.

17. Final Status

COMPLETE / PORTFOLIO READY. The project should now be frozen and published to GitHub. Further enhancement is not required for the current resume.

Recommended GitHub Description

A Flask-based resume intelligence application that compares resumes with job descriptions using weighted requirement matching, evidence analysis, skill-gap detection, recommendations, and SQLite-backed analysis

history.